

Projet Motus — application microservices

Binôme : FOMAT Gide et RIABI Hamza **M2 MIAGE SITN — Applications Web orientées Services**

Compiler et exécuter

Tout tourne en conteneurs, donc rien à installer côté Java, Maven ou PostgreSQL. Deux façons de lancer le projet :

- Avec Docker : `docker compose up --build`, puis ouvrir <http://localhost:8080>.
- Sur Minikube, en une seule commande : `bash deploy-minikube.sh`, puis <http://localhost:8080>.

Le détail (méthode manuelle de secours, ordre de démarrage, commandes `kubectl`) est dans le `README.md`.

Ce que fait l'application

On a découpé le jeu du Motus en quatre services qui se parlent en REST. Chacun a sa propre base : on peut arrêter ou modifier l'un sans toucher aux autres.

- **joueur** (8081) : inscription, connexion, rôle admin. Base `motus_joueur`.
- **partie** (8082) : le mot mystère, les essais, les règles du jeu. Base `motus_partie`.
- **score** (8083) : résultats, points, classement. Base `motus_score`.
- **frontend** (8080) : l'interface web.

Un joueur s'inscrit, choisit une longueur de mot (4 à 9 lettres) et propose des mots. Le service `partie` compare avec le mot mystère et renvoie, lettre par lettre, ce qui est bien placé, mal placé ou absent. En fin de partie, le résultat part vers `score`, qui tient le classement.

Architecture

```
frontend (8080)
├─ REST → joueur (8081) — motus_joueur
├─ REST → partie (8082) — motus_partie
└─ REST → score  (8083) — motus_score

partie → joueur  (valider le joueur avant de démarrer)
partie → score   (envoyer le résultat en fin de partie)
score  → joueur  (récupérer le pseudo pour le classement)
```

Point important : aucune base n'est partagée. Quand `partie` a besoin d'une information sur un joueur, il ne lit pas dans la base de `joueur`, il l'appelle par REST. Chaque service reste seul maître de ses tables.

Service	Port	Responsabilité	Base
joueur-service	8081	Comptes, connexion, rôle admin	motus_joueur
partie-service	8082	Dictionnaire, essais, règles	motus_partie
score-service	8083	Résultats, points, classement	motus_score

Service	Port	Responsabilité	Base
frontend-service	8080	Interface web	—

Choix techniques

- **Spring Boot 4 / Java 25**, avec Spring Web pour le REST et Spring Data JPA pour la persistance.
- **PostgreSQL, une base par service**. C'est une contrainte qu'on s'est fixée dès le départ pour que les modèles de données restent vraiment séparés.
- **REST direct entre services**, sans bus de messages ni annuaire. Vu la taille du projet, ajouter Eureka ou une passerelle aurait surtout ajouté de la complexité pour pas grand-chose.
- **Mots de passe hachés avec BCrypt**. Le rôle admin s'obtient avec un code saisi à l'inscription (`MIAGE-SITN`), ce qui suffit à distinguer un joueur d'un administrateur.
- **Dictionnaire tiré de Lexique.org**. Une grande liste sert à accepter les propositions, une liste plus courte de noms et adjectifs courants sert à tirer le mot mystère. Sans ce filtrage, on tombait sur des mots rares ou des formes conjuguées peu jouables.
- **Conteneurisation**. Un Dockerfile par service, en deux étapes (compilation puis exécution). Docker Compose pour le lancement local, puis Kubernetes/Minikube pour le déploiement (Deployment, Service, Secret pour le mot de passe, volume pour conserver les données, sondes de santé).

Modèle de données

```
Joueur ( id, pseudo, email, motDePasseHash, admin, dateInscription )

Partie ( id, joueurId, motMystere, longueur, nombreEssaisMax,
        nombreEssaisUtilises, statut, dateDebut, dateFin )
  └─ 1..* Essai ( id, ordre, mot, resultat )
    statut ∈ { EN_COURS, GAGNEE, PERDUE }

Resultat ( id, joueurId, partieId, gagnee, nombreEssais, points, date )
```

Les liens entre services (par exemple `Partie` qui référence un joueur) sont de simples identifiants. Il n'y a pas de clé étrangère d'une base vers une autre.

Bilan

Ce qu'on a aimé. Travailler sur un projet qui va du code jusqu'au déploiement réel, et pas seulement une appli qui tourne sur notre machine. Le jeu du Motus est un support agréable : les règles sont simples, mais les faire vivre à travers plusieurs services qui se coordonnent rend le tout intéressant. On a aussi aimé le côté « ça marche vraiment » du déploiement en une commande.

Ce qui a marché. On a un vrai découpage microservices, pas un monolithe déguisé : un service, une base, un schéma. Et surtout, ça se lance d'une seule commande, du poste local jusqu'au cluster Minikube. Voir l'application tourner dans Kubernetes, avec les pods qui se relancent seuls quand quelque chose tombe, c'est le moment où le cours a vraiment pris son sens pour nous.

Ce qu'on a appris. Concevoir des services qui communiquent proprement en REST, faire persister les données avec JPA sur PostgreSQL, écrire des Dockerfiles en deux étapes, et déployer sur Kubernetes (les manifestes, les Services, les Secrets, les sondes de santé). Des notions restées théoriques en cours sont devenues concrètes une fois qu'il a fallu les faire fonctionner ensemble.

Les difficultés. Elles n'étaient presque jamais dans le code du jeu, mais autour. Passer de `localhost` (qui marche en Docker Compose) aux noms de services internes de Kubernetes nous a fait chercher un bon moment. L'ordre de démarrage aussi : un service qui démarre avant que la base soit prête, ça plante — on l'a réglé avec des conteneurs d'attente. Et l'environnement Kali nous a donné du fil à retordre (installation de Maven, un problème de collation PostgreSQL après une mise à jour système).

Ce qu'on a moins aimé. Justement ça : le temps passé à configurer l'environnement, versions et ports compris, est souvent aussi long que le développement des fonctionnalités. C'est formateur, mais frustrant quand on veut juste voir le jeu tourner.